



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Práctica 3:

Circuitos de 1 y 2 orden.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Microcomputadoras

Grupo: 03 - Semestre: 2026-2

Calf: _____



1. Objetivo:

2. Desarrollo:

Los parámetros del sistema son:

- **Frecuencia de muestreo:** $f_s = 8000$ Hz
- **Período de muestreo:** $T = \frac{1}{f_s} = \frac{1}{8000} = 1.25 \times 10^{-4} \text{ s} = 125 \mu\text{s}$
- **Frecuencia de Nyquist:** $f_N = \frac{f_s}{2} = 4000$ Hz
- **Entrada analógica:** GP26 (ADC0)
- **Generación de señal de prueba:** GP3 (PWM, onda cuadrada)

Para el uso de los hilos en la pico: el **Núcleo 1** ejecuta la ecuación en diferencias del filtro activo con temporización precisa a $f_s = 8000$ Hz, mientras que el **Núcleo 0** procesa todas las entradas que se van recibiendo.

2.1. Filtro Pasa Bajas de Primer Orden (IIR Butterworth)

Se selecciona una frecuencia de corte $f_c = 500$ Hz para el filtro pasa bajas. Esto por:

1. **Criterio de Nyquist:** La frecuencia de corte debe satisfacer $f_c < f_N = 4000$ Hz, con $f_c = 500$ Hz.
2. **Relación f_c/f_s :** La relación $f_c/f_s = 500/8000 = 0.0625$ se encuentra en el rango de $(0.01 < f_c/f_s < 0.45)$, lo que hace que la transformada bilineal no introduzca distorsión por efecto de frequency warping.

Circuito eléctrico

El filtro pasa bajas analógico de primer orden se implementa con un circuito RC serie, donde la salida se toma sobre el capacitor:

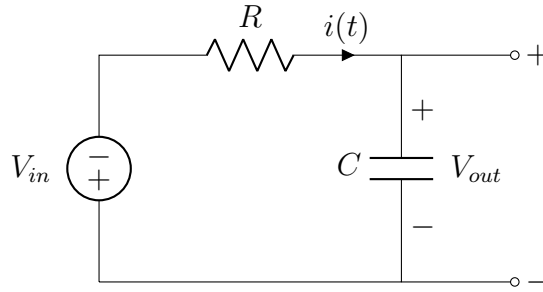


Figura 1: Circuito RC pasa bajas de primer orden.

calculo de componentes:

La frecuencia de corte del circuito RC está dada por:

$$f_c = \frac{1}{2\pi RC} \implies \omega_c = \frac{1}{RC} = 2\pi f_c \quad (1)$$

Seleccionando $C = 100$ nF (serie E24), se calcula la resistencia necesaria para obtener $f_c = 500$ Hz:

$$R = \frac{1}{2\pi f_c C} = \frac{1}{2\pi(500)(100 \times 10^{-9})} = \frac{1}{3.14159 \times 10^{-4}} = 3183.1 \, \Omega \approx 3.18 \, \text{k}\Omega \quad (2)$$

Se utiliza una resistencia de $R = 3.3 \, \text{k}\Omega$ (serie E24), lo cual resulta en una frecuencia de corte real de:

$$f_{c,\text{real}} = \frac{1}{2\pi(3300)(100 \times 10^{-9})} = \frac{1}{2.0735 \times 10^{-3}} = 482.3 \, \text{Hz} \quad (3)$$

Esta desviación de $\approx 3.5 \%$ respecto al valor teórico es aceptable.

2.1.1. Obtención de la función de transferencia $H(s)$

Aplicando la ley de Kirchhoff al circuito RC en el dominio de Laplace:

$$V_{in}(s) = R \cdot I(s) + \frac{1}{sC} \cdot I(s) = I(s) \left(R + \frac{1}{sC} \right) \quad (4)$$

El voltaje de salida sobre el capacitor es:

$$V_{out}(s) = \frac{1}{sC} \cdot I(s) \quad (5)$$



Despejando la corriente de la primera ecuación y sustituyendo:

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{\frac{1}{sC}}{R + \frac{1}{sC}} = \frac{1}{sRC + 1} \quad (6)$$

Multiplicando numerador y denominador por $\omega_c = 1/(RC)$:

$$H(s) = \frac{\omega_c}{s + \omega_c}, \quad \omega_c = 2\pi(500) = 3141.59 \text{ rad/s} \quad (7)$$

Esta es la función de transferencia de un filtro Butterworth pasa bajas de primer orden, con ganancia unitaria en DC ($H(0) = 1$) y atenuación de -3 dB en $\omega = \omega_c$.

2.1.2. Discretización mediante la transformada bilineal

Paso 1: Pre-distorsión de frecuencia. La transformada bilineal introduce una distorsión no lineal entre las frecuencias analógica y digital. Para garantizar que la frecuencia de corte sea exactamente la misma y se mantenga, se calcula la frecuencia analógica pre-distorsionada:

$$\Omega_a = \tan\left(\frac{\pi f_c}{f_s}\right) = \tan\left(\frac{\pi \cdot 500}{8000}\right) = \tan\left(\frac{\pi}{16}\right) \quad (8)$$

Evalutando:

$$\Omega_a = \tan(0.19635 \text{ rad}) = 0.19891 \quad (9)$$

$$H(s) = \frac{\Omega_a}{s + \Omega_a} \quad (10)$$

Paso 2: Sustitución de la variable bilineal.

$$s = \frac{z - 1}{z + 1} \quad (11)$$

Sustituyendo en $H(s)$:

$$H(z) = \frac{\Omega_a}{\frac{z - 1}{z + 1} + \Omega_a} \quad (12)$$

simplificando:



Multiplicando numerador y denominador por $(z + 1)$:

$$H(z) = \frac{\Omega_a(z + 1)}{(z - 1) + \Omega_a(z + 1)} \quad (13)$$

Expandiendo el denominador:

$$\begin{aligned} (z - 1) + \Omega_a(z + 1) &= z - 1 + \Omega_a z + \Omega_a \\ &= z(1 + \Omega_a) + (\Omega_a - 1) \end{aligned} \quad (14)$$

Dividiendo numerador y denominador entre $(1 + \Omega_a)$:

$$H(z) = \frac{\frac{\Omega_a}{1 + \Omega_a} (z + 1)}{z + \frac{\Omega_a - 1}{1 + \Omega_a}} = \frac{\frac{\Omega_a}{1 + \Omega_a} (z + 1)}{z - \frac{1 - \Omega_a}{1 + \Omega_a}} \quad (15)$$

Paso 3: Identificación con la forma estándar. Comparando con la forma canónica

$$H(z) = \frac{A_0 z + A_1}{z - B_1}:$$

$$A_0 = A_1 = \frac{\Omega_a}{1 + \Omega_a} = \frac{0.19891}{1 + 0.19891} = \frac{0.19891}{1.19891} = 0.16591 \quad (16)$$

$$B_1 = \frac{1 - \Omega_a}{1 + \Omega_a} = \frac{1 - 0.19891}{1 + 0.19891} = \frac{0.80109}{1.19891} = 0.66818 \quad (17)$$

Por lo tanto, la función de transferencia discreta es:

$$H(z) = \frac{0.16591 (z + 1)}{z - 0.66818} = \frac{0.16591 z + 0.16591}{z - 0.66818} \quad (18)$$

2.1.3. Ecuación en diferencias

A partir de $H(z) = Y(z)/U(z)$:

$$Y(z)(z - B_1) = U(z)(A_0 z + A_1) \quad (19)$$

Dividiendo ambos lados entre z :

$$Y(z) - B_1 z^{-1} Y(z) = A_0 U(z) + A_1 z^{-1} U(z) \quad (20)$$

Aplicando la transformada z inversa (donde z^{-1} representa un retardo de una muestra):

$$y(k) = A_0 \cdot u(k) + A_1 \cdot u(k-1) + B_1 \cdot y(k-1) \quad (21)$$

Sustituyendo los valores numéricos:

$$y(k) = 0.16591 \cdot u(k) + 0.16591 \cdot u(k-1) + 0.66818 \cdot y(k-1) \quad (22)$$

Coefficientes:

Tabla 1: Coeficientes del filtro pasa bajas de primer orden ($f_c = 500$ Hz, $f_s = 8000$ Hz).

Coeficiente	Expresión	Valor numérico
A_0	$\frac{\Omega_a}{1 + \Omega_a}$	0.16591
A_1	$\frac{\Omega_a}{1 + \Omega_a}$	0.16591
A_2	—	0.0
B_1	$\frac{1 - \Omega_a}{1 + \Omega_a}$	0.66818
B_2	—	0.0

Verificación en DC ($z = 1$):

$$H(1) = \frac{A_0 + A_1}{1 - B_1} = \frac{0.16591 + 0.16591}{1 - 0.66818} = \frac{0.33182}{0.33182} = 1.0 \quad (23)$$

De este modo, se verifica que la componente DC no sufre atenuación.

2.1.4. Diagrama de flujo de señal

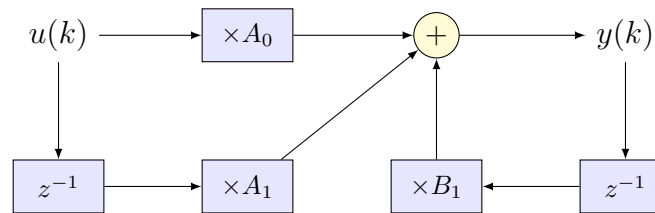


Figura 2: Diagrama de flujo de señal del filtro pasa bajas de primer orden (Forma Directa I).



2.2. Diseño del Filtro Pasa Altas de Segundo Orden (Circuito RLC)

Se selecciona una frecuencia de corte $f_c = 800$ Hz para el filtro pasa altas de segundo orden. Para esto, se toma en cuenta lo siguiente:

1. **Criterio de Nyquist:** Se cumple $f_c = 800$ Hz $< f_N = 4000$ Hz.
2. **Relación:** La relación $f_c/f_s = 800/8000 = 0.1$ se ubica en la zona central del rango recomendado, donde la transformada bilineal presenta mínima distorsión.

Circuito:

El filtro pasa altas de segundo orden se implementa obligatoriamente con un circuito **RLC serie**, ya que para obtener una función de transferencia de segundo grado se requieren al menos dos elementos almacenadores (capacitor e inductor).

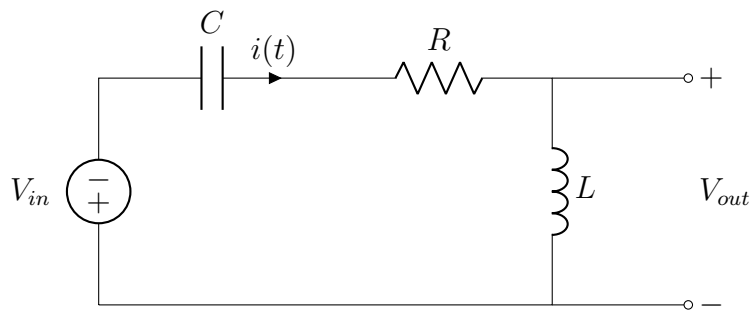


Figura 3: Circuito RLC serie pasa altas de segundo orden (salida sobre el inductor L).

calculo de los componentes:

Del circuito RLC serie se obtiene:

$$H(s) = \frac{s^2}{s^2 + \frac{R}{L}s + \frac{1}{LC}} \quad (24)$$

Comparando con la forma canónica Butterworth de segundo orden:

$$H(s) = \frac{s^2}{s^2 + \sqrt{2}\omega_c s + \omega_c^2} \quad (25)$$



Se identifican las relaciones de diseño:

$$\frac{1}{LC} = \omega_c^2, \quad \frac{R}{L} = \sqrt{2}\omega_c \quad (26)$$

Con $\omega_c = 2\pi(800) = 5026.55$ rad/s y $L = 100$ mH:

$$C = \frac{1}{L \cdot \omega_c^2} = \frac{1}{0.1 \times (5026.55)^2} = \frac{1}{0.1 \times 25,266,219} = \frac{1}{2,526,622}$$
$$C = 3.958 \times 10^{-7} \text{ F} = 395.8 \text{ nF} \approx 390 \text{ nF} \quad (27)$$

$$R = \sqrt{2}\omega_c \cdot L = 1.41421 \times 5026.55 \times 0.1$$
$$R = 710.8 \Omega \quad (28)$$

2.2.1. Obtención de la función de transferencia $H(s)$

Aplicando la LVK al circuito RLC serie en el dominio de Laplace:

$$V_{in}(s) = \frac{1}{sC} I(s) + R I(s) + sL I(s) = I(s) \left(\frac{1}{sC} + R + sL \right) \quad (29)$$

La salida se toma sobre el inductor:

$$V_{out}(s) = sL \cdot I(s) \quad (30)$$

La función de transferencia es:

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{sL}{\frac{1}{sC} + R + sL} \quad (31)$$

Multiplicando numerador y denominador por sC :

$$H(s) = \frac{s^2 LC}{s^2 LC + sRC + 1} \quad (32)$$



Dividiendo entre LC :

$$H(s) = \frac{s^2}{s^2 + \frac{R}{L}s + \frac{1}{LC}} = \frac{s^2}{s^2 + \sqrt{2}\omega_c s + \omega_c^2} \quad (33)$$

donde $\omega_c = 2\pi(800) = 5026.55$ rad/s. La ganancia es cero en DC ($H(0) = 0$, bloquea la componente continua) y unitaria en alta frecuencia ($\lim_{s \rightarrow \infty} H(s) = 1$), confirmando el comportamiento pasa altas.

2.2.2. Discretización mediante la transformada bilineal (Tustin)

Paso 1: Pre-distorsión de frecuencia.

$$\Omega_a = \tan\left(\frac{\pi f_c}{f_s}\right) = \tan\left(\frac{\pi \cdot 800}{8000}\right) = \tan\left(\frac{\pi}{10}\right) = \tan(18) = 0.32492 \quad (34)$$

$$H(s) = \frac{s^2}{s^2 + \sqrt{2}\Omega_a s + \Omega_a^2} \quad (35)$$

Paso 2: Sustitución bilineal $s = \frac{z-1}{z+1}$.

$$H(z) = \frac{\left(\frac{z-1}{z+1}\right)^2}{\left(\frac{z-1}{z+1}\right)^2 + \sqrt{2}\Omega_a \left(\frac{z-1}{z+1}\right) + \Omega_a^2} \quad (36)$$

Simplificación:

Multiplicando numerador y denominador por $(z+1)^2$:

$$H(z) = \frac{(z-1)^2}{(z-1)^2 + \sqrt{2}\Omega_a (z-1)(z+1) + \Omega_a^2 (z+1)^2} \quad (37)$$

Expandiendo cada término del denominador por separado:

$$(z-1)^2 = z^2 - 2z + 1 \quad (38)$$

$$\sqrt{2}\Omega_a (z-1)(z+1) = \sqrt{2}\Omega_a (z^2 - 1) = 0.45944 z^2 - 0.45944 \quad (39)$$

$$\Omega_a^2 (z+1)^2 = 0.10557 (z^2 + 2z + 1) = 0.10557 z^2 + 0.21114 z + 0.10557 \quad (40)$$



Agrupando el denominador por potencias:

$$D(z) = z^2 \underbrace{(1 + 0.45944 + 0.10557)}_{D_0} + z \underbrace{(-2 + 0 + 0.21114)}_{D_1} + \underbrace{(1 - 0.45944 + 0.10557)}_{D_2} \quad (41)$$

Evalando:

$$D_0 = 1 + \sqrt{2} \Omega_a + \Omega_a^2 = 1 + 0.45944 + 0.10557 = 1.56501 \quad (42)$$

$$D_1 = -2 + 2\Omega_a^2 = -2 + 2(0.10557) = -2 + 0.21114 = -1.78886 \quad (43)$$

$$D_2 = 1 - \sqrt{2} \Omega_a + \Omega_a^2 = 1 - 0.45944 + 0.10557 = 0.64613 \quad (44)$$

Paso 4: Normalizando: Dividiendo numerador y denominador entre $D_0 = 1.56501$:

$$H(z) = \frac{\frac{1}{D_0} (z^2 - 2z + 1)}{z^2 + \frac{D_1}{D_0} z + \frac{D_2}{D_0}} \quad (45)$$

Calculando cada coeficiente:

$$A_0 = \frac{1}{D_0} = \frac{1}{1.56501} = 0.63897 \quad (46)$$

$$A_1 = \frac{-2}{D_0} = \frac{-2}{1.56501} = -1.27795 \quad (47)$$

$$A_2 = \frac{1}{D_0} = \frac{1}{1.56501} = 0.63897 \quad (48)$$

Para los coeficientes del denominador, se utiliza la convención de la ecuación en diferencias $y(k) = \dots + B_1 y(k-1) + B_2 y(k-2)$:

$$B_1 = -\frac{D_1}{D_0} = -\frac{-1.78886}{1.56501} = 1.14305 \quad (49)$$

$$B_2 = -\frac{D_2}{D_0} = -\frac{0.64613}{1.56501} = -0.41286 \quad (50)$$

La función de transferencia discreta resulta:

$$H(z) = \frac{0.63897 z^2 - 1.27795 z + 0.63897}{z^2 - 1.14305 z + 0.41286} \quad (51)$$



2.2.3. Ecuación en diferencias

A partir de $H(z) = Y(z)/U(z)$:

$$Y(z) (z^2 - B_1 z - B_2) = U(z) (A_0 z^2 + A_1 z + A_2) \quad (52)$$

Dividiendo ambos lados entre z^2 y aplicando la transformada z inversa:

$$y(k) - B_1 y(k-1) - B_2 y(k-2) = A_0 u(k) + A_1 u(k-1) + A_2 u(k-2) \quad (53)$$

Reorganizando:

$$\boxed{y(k) = A_0 u(k) + A_1 u(k-1) + A_2 u(k-2) + B_1 y(k-1) + B_2 y(k-2)} \quad (54)$$

Sustituyendo los valores numéricos:

$$y(k) = 0.63897 u(k) - 1.27795 u(k-1) + 0.63897 u(k-2) \\ + 1.14305 y(k-1) - 0.41286 y(k-2) \quad (55)$$

coeficientes:

Tabla 2: Coeficientes del filtro pasa altas de segundo orden ($f_c = 800$ Hz, $f_s = 8000$ Hz).

Coeficiente	Expresión	Valor numérico
A_0	$\frac{1}{D_0}$	0.63897
A_1	$\frac{-2}{D_0}$	-1.27795
A_2	$\frac{1}{D_0}$	0.63897
B_1	$-\frac{D_1}{D_0}$	1.14305
B_2	$-\frac{D_2}{D_0}$	-0.41286

1. **En DC** ($z = 1$):

$$H(1) = \frac{A_0 + A_1 + A_2}{1 - B_1 - B_2} = \frac{0.63897 - 1.27795 + 0.63897}{1 - 1.14305 + 0.41286} = \frac{0}{0.26981} = 0 \quad (56)$$

Esto confirma que el filtro pasa altas bloquea completamente la componente DC.

2. **En Nyquist** ($z = -1$):

$$H(-1) = \frac{A_0 - A_1 + A_2}{1 + B_1 - B_2} = \frac{0.63897 + 1.27795 + 0.63897}{1 + 1.14305 + 0.41286} = \frac{2.55589}{2.55591} \approx 1.0 \quad (57)$$

Esto confirma que la ganancia es unitaria en la frecuencia de Nyquist.

3. **Suma de coeficientes del numerador:**

$$A_0 + A_1 + A_2 = 0.63897 - 1.27795 + 0.63897 = 0 \quad (58)$$

garantiza $H(1) = 0$.

2.2.4. Diagrama de flujo de señal

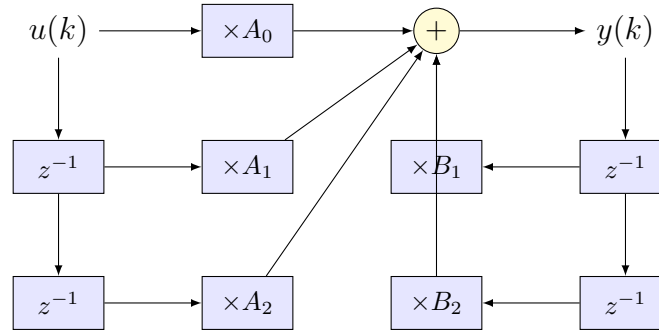


Figura 4: Diagrama de flujo de señal del filtro pasa altas de segundo orden (Forma Directa I).



2.3. Implementación en MicroPython

2.3.1. Documentación del código

El programa comienza importando las bibliotecas necesarias para interactuar con el hardware de la Raspberry Pi Pico 2W. De la biblioteca `machine` se traen las clases `Pin`, `ADC` y `PWM`, que permiten configurar pines digitales, leer señales analógicas y generar ondas con modulación de ancho de pulso, respectivamente. También se importa el módulo `_thread`, que es la herramienta que ofrece MicroPython para trabajar con los dos núcleos del procesador RP2350 de forma simultánea, y el módulo `time`, que se utiliza para medir tiempos y hacer pausas con precisión de microsegundos.

A continuación se definen los parámetros fundamentales del sistema. La frecuencia de muestreo se fija en 8000 Hz, lo que implica que el filtro debe calcular una nueva muestra cada 125 microsegundos. Estos valores se guardan en las constantes `FS` y `PERIOD_US` para que sean fáciles de modificar si en algún momento se quisiera cambiar la tasa de muestreo.

Después viene la configuración del hardware. El pin GP26 se configura como entrada analógica a través de la clase `ADC`, ya que es uno de los pines que tiene convertidor analógico-digital integrado en la Pico. El pin GP3 se configura como salida PWM para generar la onda cuadrada que sirve como señal de prueba; se le asigna una frecuencia inicial de 200 Hz y un ciclo de trabajo del 50 %, lo que produce una onda simétrica. Finalmente, el LED integrado de la placa se configura como salida digital para servir como indicador visual de que el sistema se encuentra filtrando activamente.

Los coeficientes de los dos filtros se declaran como constantes globales. Para el filtro pasa bajas de primer orden, que se diseñó a partir del circuito RC con frecuencia de corte de 500 Hz, los coeficientes son $A_0 = A_1 = 0.16591$ y $B_1 = 0.66818$, obtenidos mediante la transformada bilineal como se mostró en la sección anterior. Para el filtro pasa altas de segundo orden, derivado del circuito RLC con frecuencia de corte de 800 Hz, los coeficientes son $A_0 = 0.63897$, $A_1 = -1.27795$, $A_2 = 0.63897$, $B_1 = 1.14305$ y $B_2 = -0.41286$. Estos valores se almacenan con cinco decimales para mantener una precisión adecuada en los cálculos de punto flotante.

Luego se declaran las variables de estado que necesita cada filtro para recordar las muestras anteriores. El filtro pasa bajas solo requiere guardar la entrada anterior $u(k-1)$ y la salida anterior $y(k-1)$, ya que es de primer orden. El filtro pasa altas, al ser de segundo orden, necesita dos entradas pasadas $u(k-1)$ y $u(k-2)$, además de dos salidas pasadas $y(k-1)$



y $y(k - 2)$. Todas estas variables se inicializan en cero. También se crean dos variables de control: **active_filter**, que indica cuál filtro está seleccionado, y **running**, que determina si el sistema está procesando muestras o se encuentra detenido.

Se define una función auxiliar llamada **mostrar_menu** que imprime por el puerto serial un encabezado con el nombre del sistema, la frecuencia de muestreo, y la lista de comandos disponibles. Esta función se llama al inicio del programa y cada vez que el usuario detiene el filtrado, para que siempre tenga a la vista las opciones que puede utilizar.

La parte central del programa es la función **filter_core**, que se ejecuta en el Núcleo 1 del procesador. Esta función corre en un ciclo infinito y, mientras la variable **running** sea verdadera, realiza lo siguiente en cada iteración: primero registra el tiempo de inicio para poder controlar la duración del ciclo; luego lee el valor del ADC, que devuelve un número entero de 16 bits (entre 0 y 65535), y lo convierte a voltaje multiplicando por 3.3 y dividiendo entre 65535. Una vez obtenida la muestra de entrada $u(k)$ en voltios, se evalúa cuál filtro está activo. Si es el pasa bajas, se calcula la salida aplicando la ecuación en diferencias $y(k) = A_0 \cdot u(k) + A_1 \cdot u(k - 1) + B_1 \cdot y(k - 1)$ y se actualizan las variables de estado. Si es el pasa altas, se usa la ecuación completa de segundo orden con los cinco coeficientes y se actualizan las cuatro variables de estado, teniendo cuidado de respetar el orden de asignación para no sobrescribir valores que todavía se necesitan. Después de calcular la salida, se envía por el puerto serial una línea con el formato **entrada,salida**, que es el formato que espera el Serial Plotter. Al final del ciclo se mide cuánto tiempo transcurrió y, si fue menos de 125 microsegundos, se espera el tiempo restante para mantener la frecuencia de muestreo constante. Si el cálculo tomó más tiempo del disponible, se hace una pausa mínima de cero microsegundos para ceder brevemente el procesador al otro núcleo.

La función **filter_core** se lanza en el Núcleo 1 mediante la instrucción **_thread.start_new_thread**, lo que permite que ambos núcleos del RP2350 trabajen de manera simultánea sin interferirse. A partir de ese momento, el Núcleo 1 queda dedicado exclusivamente al cálculo del filtro.

El Núcleo 0 se encarga de la interacción con el usuario a través del puerto serial USB. Tras mostrar el menú inicial, entra en un ciclo infinito donde espera a recibir un comando. La función **input()** lee una línea completa del puerto serial, y después se convierte a mayúsculas para que el usuario pueda escribir los comandos en cualquier combinación de mayúsculas y minúsculas. Según el comando recibido, el programa ejecuta la acción correspondiente: **START** activa el filtrado y enciende el LED; **STOP** lo detiene, apaga el LED y vuelve a mostrar el menú;



LP selecciona el filtro pasa bajas y reinicia sus variables de estado; HP hace lo mismo para el filtro pasa altas; **FREQ** seguido de un número cambia la frecuencia de la onda cuadrada de prueba que genera el PWM; y **STATUS** muestra en pantalla qué filtro está seleccionado, si está activo o detenido, la frecuencia de muestreo y la frecuencia actual de la onda cuadrada. Si el comando no corresponde a ninguna opción reconocida, se muestra un mensaje de error. Todo el bloque de lectura y procesamiento de comandos se encuentra dentro de un **try-except** para que, si ocurre algún error inesperado en la comunicación serial, el programa no se detenga sino que simplemente espere un breve instante y vuelva a intentar leer.

2.3.2. Código fuente

Listing 1: Código MicroPython del sistema de filtros digitales IIR para Raspberry Pi Pico 2W.

```
1 # =====
2 # filtros_iir.py
3 # Sistema de Filtros Digitales IIR - Raspberry Pi Pico 2W
4 #
5 # Filtro 1: Pasa Bajas de 1er Orden (IIR Butterworth, fc=500 Hz)
6 # Filtro 2: Pasa Altas de 2do Orden (Circuito RLC, fc=800 Hz)
7 #
8 # fs = 8000 Hz | T = 125 us
9 # Comunicación: USB Serial (COM5)
10 #
11 # Arquitectura de doble núcleo:
12 #   Núcleo 0: Comunicación serial y comandos
13 #   Núcleo 1: Ecuación en diferencias del filtro
14 # =====
15
16 from machine import Pin, ADC, PWM
17 import _thread
18 import time
19
20 # =====
21 # PARÁMETROS DEL SISTEMA
22 # =====
23 FS = 8000 # Frecuencia de muestreo (Hz)
```



```
24 PERIOD_US = 125                                # Período de muestreo (us)
25
26 # =====
27 # CONFIGURACIÓN DE HARDWARE
28 # =====
29 adc = ADC(Pin(26))                             # GP26 = ADC0 (entrada analógica)
30 pwm_sq = PWM(Pin(3))                           # GP3 = Onda cuadrada de prueba
31 pwm_sq.freq(200)                               # Frecuencia inicial: 200 Hz
32 pwm_sq.duty_u16(32768)                         # 50% duty cycle
33 led = Pin("LED", Pin.OUT)                     # LED indicador
34
35 # =====
36 # COEFICIENTES - FILTRO PASA BAJAS (1er Orden, Butterworth)
37 # Circuito: RC serie (R=3.18k, C=100nF)
38 # fc = 500 Hz | Omega_a = tan(pi*500/8000) = 0.19891
39 #  $H(z) = 0.16591*(z + 1) / (z - 0.66818)$ 
40 #  $y(k) = A0*u(k) + A1*u(k-1) + B1*y(k-1)$ 
41 # =====
42 LP_A0 = 0.16591
43 LP_A1 = 0.16591
44 LP_B1 = 0.66818
45
46 # =====
47 # COEFICIENTES - FILTRO PASA ALTAS (2do Orden, RLC Butterworth)
48 # Circuito: RLC serie (C=390nF, R=680ohm, L=100mH), salida en L
49 # fc = 800 Hz | Omega_a = tan(pi*800/8000) = 0.32492
50 #  $H(z) = (0.63897*z^2 - 1.27795*z + 0.63897) /$ 
51 #  $(z^2 - 1.14305*z + 0.41286)$ 
52 #  $y(k) = A0*u(k) + A1*u(k-1) + A2*u(k-2) + B1*y(k-1) + B2*y(k-2)$ 
53 # =====
54 HP_A0 = 0.63897
55 HP_A1 = -1.27795
56 HP_A2 = 0.63897
57 HP_B1 = 1.14305
58 HP_B2 = -0.41286
59
```



```
60 # =====
61 # VARIABLES DE ESTADO
62 # =====
63 # Pasa bajas (1er orden)
64 lp_u1 = 0.0          # u(k-1)
65 lp_y1 = 0.0          # y(k-1)
66
67 # Pasa altas (2do orden)
68 hp_u1 = 0.0          # u(k-1)
69 hp_u2 = 0.0          # u(k-2)
70 hp_y1 = 0.0          # y(k-1)
71 hp_y2 = 0.0          # y(k-2)
72
73 # Control
74 active_filter = 0     # 0 = Pasa Bajas, 1 = Pasa Altas
75 running = False
76
77 # =====
78 # FUNCIÓN DEL MENÚ
79 # =====
80 def mostrar_menu():
81     print("\n" + "=" * 50)
82     print(" Sistema de Filtros Digitales IIR")
83     print(" Raspberry Pi Pico 2W | fs = {} Hz".format(FS))
84     print("=" * 50)
85     print("Comandos disponibles:")
86     print("  START      - Iniciar filtrado")
87     print("  STOP       - Detener filtrado")
88     print("  LP         - Filtro pasa bajas (fc=500 Hz)")
89     print("  HP         - Filtro pasa altas (fc=800 Hz)")
90     print("  FREQ <hz>  - Frecuencia de onda cuadrada")
91     print("  STATUS     - Estado actual del sistema")
92     print("=" * 50)
93
94 # =====
95 # NÚCLEO 1: ECUACIÓN EN DIFERENCIAS (ejecuta a fs = 8000 Hz)
```



```
96 # =====
97 def filter_core():
98     global lp_u1, lp_y1
99     global hp_u1, hp_u2, hp_y1, hp_y2
100    global active_filter, running
101
102    while True:
103        if not running:
104            time.sleep_ms(1)
105            continue
106
107        t0 = time.ticks_us()
108
109        # Lectura del ADC (16 bits: 0-65535)
110        raw = adc.read_u16()
111        u_k = (raw / 65535.0) * 3.3    # Conversión a voltaje (0-3.3
            V)
112
113        if active_filter == 0:
114            # ---- Filtro Pasa Bajas 1er Orden (RC) ----
115            # y(k) = A0*u(k) + A1*u(k-1) + B1*y(k-1)
116            y_k = LP_A0 * u_k + LP_A1 * lp_u1 + LP_B1 * lp_y1
117            # Actualizar estados
118            lp_u1 = u_k
119            lp_y1 = y_k
120        else:
121            # ---- Filtro Pasa Altas 2do Orden (RLC) ----
122            # y(k) = A0*u(k) + A1*u(k-1) + A2*u(k-2)
123            #           + B1*y(k-1) + B2*y(k-2)
124            y_k = (HP_A0 * u_k + HP_A1 * hp_u1 + HP_A2 * hp_u2
125                  + HP_B1 * hp_y1 + HP_B2 * hp_y2)
126            # Actualizar estados (orden importa)
127            hp_u2 = hp_u1
128            hp_u1 = u_k
129            hp_y2 = hp_y1
130            hp_y1 = y_k
```



```
131
132     # Enviar datos por USB Serial: entrada, salida
133     print("{:.4f},{:.4f}".format(u_k, y_k))
134
135     # Esperar resto del período de muestreo
136     elapsed = time.ticks_diff(time.ticks_us(), t0)
137     if elapsed < PERIOD_US:
138         time.sleep_us(PERIOD_US - elapsed)
139     else:
140         # Forzamos al Núcleo 1 a ceder el procesador (GIL) por
141         un instante.
142         # Esto evita que el Núcleo 0 se congele si el ciclo toma
143         más de 125us.
144         time.sleep_us(0)
145
146     # =====
147     # INICIAR NÚCLEO 1
148     # =====
149     _thread.start_new_thread(filter_core, ())
150
151     # =====
152     # NÚCLEO 0: COMUNICACIÓN SERIAL Y COMANDOS
153     # =====
154     mostrar_menu()
155
156     while True:
157         try:
158             line = input()                # Lectura bloqueante (Core
159             0)
160             cmd = line.strip().upper()
161
162             if cmd == "START":
163                 running = True
164                 led.on()
165                 print("OK: Filtrado iniciado")
```



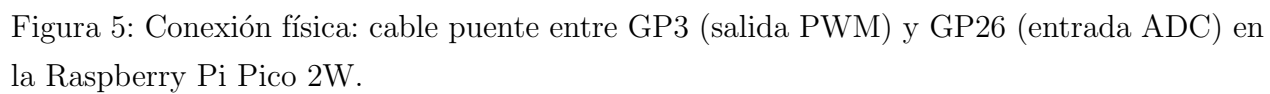
```
164 elif cmd == "STOP":
165     running = False
166     led.off()
167     print("OK: Filtrado detenido")
168     mostrar_menu()          # Se vuelve a mostrar el men
                             ú inmediatamente
169
170 elif cmd == "LP":
171     active_filter = 0
172     lp_u1 = 0.0
173     lp_y1 = 0.0
174     print("OK: Pasa Bajas 1er Orden (fc=500 Hz)")
175
176 elif cmd == "HP":
177     active_filter = 1
178     hp_u1 = 0.0
179     hp_u2 = 0.0
180     hp_y1 = 0.0
181     hp_y2 = 0.0
182     print("OK: Pasa Altas 2do Orden (fc=800 Hz)")
183
184 elif cmd.startswith("FREQ"):
185     parts = cmd.split()
186     if len(parts) >= 2:
187         try:
188             freq = int(parts[1])
189             pwm_sq.freq(freq)
190             print("OK: Onda cuadrada = {} Hz".format(freq))
191         except ValueError:
192             print("ERR: Frecuencia no valida")
193     else:
194         print("Uso: FREQ <frecuencia_hz>")
195
196 elif cmd == "STATUS":
197     nombre = "Pasa Bajas (fc=500Hz)" if active_filter == 0 \
198         else "Pasa Altas (fc=800Hz)"
```



```
199         estado = "Activo" if running else "Detenido"
200         frecuencia_onda = pwm_sq.freq() # Leemos la frecuencia
           actual del hardware
201
202         print("Filtro: {} | Estado: {} | fs: {} Hz | Onda
           Cuadrada: {} Hz".format(
203             nombre, estado, FS, frecuencia_onda))
204
205     elif cmd != "":
206         print("ERR: Comando desconocido: {}".format(line))
207
208     except Exception as e:
209         time.sleep_ms(100)
```

2.3.3. Diagrama de conexión física

Para probar el sistema, la señal de prueba generada por el pin GP3 (PWM) se conecta directamente al pin GP26 (ADC0) mediante un cable puente. De esta forma, la onda cuadrada producida por el propio microcontrolador entra al convertidor analógico-digital sin necesidad de componentes externos, lo que permite verificar el funcionamiento de los filtros digitales de manera sencilla.





3. Resultados:

3.1. coeficientes

Tabla 3: Comparación de coeficientes de los filtros diseñados.

Coeficiente	Pasa Bajas 1er Orden	Pasa Altas 2do Orden
A_0	0.16591	0.63897
A_1	0.16591	-1.27795
A_2	0.0	0.63897
B_1	0.66818	1.14305
B_2	0.0	-0.41286
f_c	500 Hz	800 Hz
Tipo	IIR Butterworth	RLC Butterworth
Orden	1	2



3.1.1. Valores esperados con señal de onda cuadrada

Al aplicar una onda cuadrada de frecuencia f_{sq} como señal de prueba, se esperan los siguientes comportamientos:

Tabla 4: Comportamiento esperado con onda cuadrada a distintas frecuencias.

f_{sq} (Hz)	Pasa Bajas ($f_c = 500$ Hz)	Pasa Altas ($f_c = 800$ Hz)
50 Hz	La salida sigue la onda cuadrada con esquinas redondeadas (atenuación mínima).	La salida muestra picos en cada transición, decayendo a cero entre flancos.
200 Hz	Onda cuadrada visible con suavizado moderado de las transiciones.	Picos pronunciados en los flancos; la componente fundamental se atenúa parcialmente.
500 Hz	Señal significativamente suavizada; la fundamental pasa con -3 dB.	Picos visibles; la fundamental pasa parcialmente (-8.9 dB de atenuación).
1000 Hz	Señal muy atenuada y suavizada, salida casi triangular/senoidal.	La onda cuadrada pasa con modificación moderada (-1.5 dB en la fundamental).
2000 Hz	Salida fuertemente atenuada (-12.3 dB); casi imperceptible.	La onda cuadrada pasa prácticamente sin modificación (-0.1 dB).



4. Conclusiones:

- **Espinoza Matamoros Percival Ulises:** A partir de la realización de esta práctica pude retomar conceptos vistos en asignaturas anteriores, de forma que a partir de diseñar un filtro y a partir de esta obtener sus funciones de transferencia, al tener que implementar el filtro en la Raspberry Pi Pico, pude aprender y comprender la importancia así como la necesidad de discretizar la función de transferencia, para así poder implementarla en un microcontrolador, donde por medio de una señal cuadrada de entrada se pudo observar el comportamiento del filtro, comprobando así que el filtro se comporta de la forma esperada, es decir, que se atenúan las frecuencias bajas para el caso del filtro pasa altas, y se atenúan las frecuencias altas para el caso del filtro pasa bajas. Por otro lado al implementar estos dos filtros se pudo observar que el filtro de primer orden tiene una respuesta más lenta, es decir, que tarda más en estabilizarse, mientras que el filtro de segundo orden tiene una respuesta más rápida, es decir, que tarda menos en estabilizarse. De forma que en la raspberry pi pico se pueden implementar filtros digitales de forma sencilla, y con un buen desempeño, siempre y cuando se tenga una buena discretización de la función de transferencia y se realicen los cálculos correspondientes de forma correcta.
- **Flores Colin Victor Jaziel:**
- **Garcia Cortes Adolfo de Jesus:**
- **Lara Hernandez Angel Husiel:**
- **Lugo Manzano Rodrigo:**



Referencias

- Butterworth, S. (1930). On the Theory of Filter Amplifiers. *Wireless Engineer*, 7(6), 536-541.
- MicroPython Contributors. (2024a). *_thread — Multithreading Support*. https://docs.micropython.org/en/latest/library/_thread.html
- MicroPython Contributors. (2024b). *class ADC — Analog to Digital Conversion*. <https://docs.micropython.org/en/latest/library/machine.ADC.html>
- MicroPython Contributors. (2024c). *class PWM — Pulse Width Modulation*. <https://docs.micropython.org/en/latest/library/machine.PWM.html>
- MicroPython Contributors. (2024d). *machine — Functions Related to the Hardware*. <https://docs.micropython.org/en/latest/library/machine.html>
- MicroPython Contributors. (2024e). *time — Time Related Functions*. <https://docs.micropython.org/en/latest/library/time.html>
- Ogata, K. (2010). *Modern Control Engineering* (5.^a ed.). Pearson.
- Python Software Foundation. (2024). *_thread — Low-Level Threading API*. https://docs.python.org/3/library/_thread.html
- Raspberry Pi Ltd. (2024a). *Raspberry Pi Pico 2 W Datasheet*. <https://datasheets.raspberrypi.com/pico/pico-2-w-datasheet.pdf>
- Raspberry Pi Ltd. (2024b). *RP2350 Datasheet: A Microcontroller by Raspberry Pi*. <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- Sedra, A. S., Smith, K. C., Carusone, T. C., & Gaudet, V. (2020). *Microelectronic Circuits* (8.^a ed.). Oxford University Press.